

Вывод данных, команда print

Для вывода данных на экран используется команда `print()`.

Внутри круглых скобок пишем, что хотим вывести на экран. Если это текст, то обязательно указываем его внутри кавычек. Кавычки могут быть одинарными или двойными. Только обязательно ставим одинаковые до и после текста.

Например, следующий код:

```
print('Мы изучаем язык Python')
```

выведет на экран текст:

```
Мы изучаем язык Python
```

Запомни: кавычки могут быть и одинарными, и двойными. Следующие две строки делают одно и то же.

```
print('Python')
```

```
print("Python")
```

То, что мы пишем в круглых скобках у команды `print()`, называется **аргументами** или **параметрами** команды.

Команда `print()` позволяет указывать несколько аргументов, в таком случае **их надо отделять запятыми**. Если вы не будете писать запятые между аргументами, Python воспримет это как синтаксическую ошибку.

Например, следующий код:

```
print('Скоро я', 'буду программировать', 'на языке', 'Python!')
```

выведет на экран текст:

```
Скоро я_буду программировать_на языке_Python!
```

Обратите внимание, в качестве разделителя при выводе данных между аргументами команды используется **символ пробел**. По умолчанию команда `print()` добавляет ровно один пробел между всеми своими аргументами. Например, следующая программа:

```
print('1', '2', '4', '8', '16')
```

выводит числа `1 2 4 8 16`. Числа выведены ровно через один пробел.

Запомни: при написании кода между аргументами команды `print()` после запятой мы ставим 1 символ пробел. Это общепринятое соглашение в языке Python. Этот символ пробела не влияет на вывод данных.

Примечания

Примечание 1. Команда `print()` записывается только маленькими буквами, другое написание недопустимо, так как в Python строчные и заглавные буквы [различны](#).

Примечание 2. Каждая последующая команда `print()` выводит указанный текст **с новой строки**. Например, следующая программа:

```
print('Какой хороший день!')  
print('Работать мне не лень!')
```

выведет на экран две строки:

```
Какой хороший день!  
Работать мне не лень!
```

Примечание 3. Команда `print()` с пустым списком аргументов просто вставляет новую пустую строку. Например:

```
print('Какой хороший день!')  
  
print('Работать мне не лень!')
```

выведет на экран три строки, одна из которых пустая:

```
Какой хороший день!  
  
Работать мне не лень!
```

Примечание 4. Почему в Python можно использовать как одинарные, так и двойные кавычки для обрамления текста? Ответ очень прост — часто кавычки это часть текста. И чтобы Python мог правильно распознать такой текст, пользуемся разными:

- если в тексте нужны одинарные кавычки, то для обрамления такого текста используем двойные кавычки;
- если в тексте нужны двойные кавычки, то обрамляем его одинарными.

Результатом выполнения кода:

```
print('В тексте есть "двойные" кавычки')  
print("В тексте есть 'одинарные' кавычки")
```

будет:

В тексте есть "двойные" кавычки

В тексте есть 'одинарные' кавычки

Ввод данных, команда input

Все предыдущие программы выводили на экран текст, известный в момент написания программного кода. Однако программы могут работать с данными, которые станут известны только во время выполнения программы. Другими словами, программы могут считывать данные, а затем их использовать.

Для считывания данных в языке Python используется команда `input()`.

Рассмотрим следующую программу:

```
print('Как тебя зовут?')
```

```
name = input()
```

```
print('Привет, ', name)
```

Сначала программа распечатает текст на экран «Как тебя зовут?». Далее программа будет ждать от пользователя ввода данных. Ввод данных реализуется с помощью команды `input()`.

Команда `input()` всегда пишется с круглыми скобками. Она работает так: когда программа доходит до места, где есть `input()`, она ждет, пока пользователь введёт текст с клавиатуры (ввод завершается нажатием клавиши Enter). Введенная строка подставляется на место `input()`.

То есть, если вы ввели строку «Parrot», программа дальше будет работать так, как будто на месте `input()` было написано «Parrot».

Таким образом, `input()` получает от пользователя какие-то данные и вместо вызова подставляет **строковое значение**, в нашем случае записывает его в качестве значения **переменной** `name`.

Что такое переменные и что значит сохранить в переменной значение, подробнее поговорим чуть позже, однако для решения следующих задач используйте переменные, которые состоят из латиницы, не содержат внутри себя пробелов, могут включать символ нижнего подчёркивания. Например, такие названия хорошие:

```
name, age, team, string, string_1, string_2, string_666
```

Запоминаем:

Команда `print()` выводит на экран данные.

Команда `input()` считывает введенные с клавиатуры данные.

Примечания

Примечание. Очень часто перед считыванием данных мы печатаем некоторый текст, чтобы пользователь, который вводит эти данные, понимал, что именно от него требуется. Например, в программе

```
print('Как тебя зовут?')
```

```
name = input()
```

```
print('Привет, ', name)
```

мы сначала выведем текст «Как тебя зовут?», а уже потом считаем данные.

Поскольку это достаточно распространённый сценарий, то в языке Python можно выводить текст, передавая его в качестве параметра в команду `input()`.

Предыдущий код можно переписать так:

```
# сначала тут печатается строка "Как тебя зовут", а потом принимается  
на вход имя
```

```
name = input('Как тебя зовут?')
```

```
# тут просто выводится строка "Привет", после неё идёт пробел и  
введённое нами имя
```

```
print('Привет, ', name)
```

Однако в задачах нашего курса нужно использовать функцию `input()` **без параметров!** Так как это будет лишний вывод, ваша программа не будет проходить тесты.